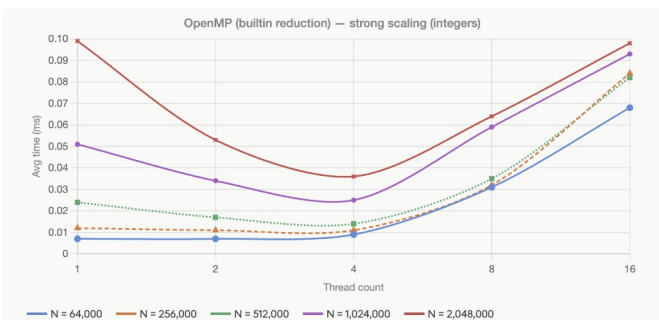
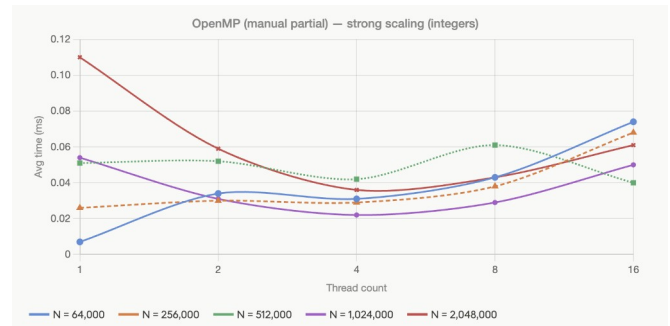
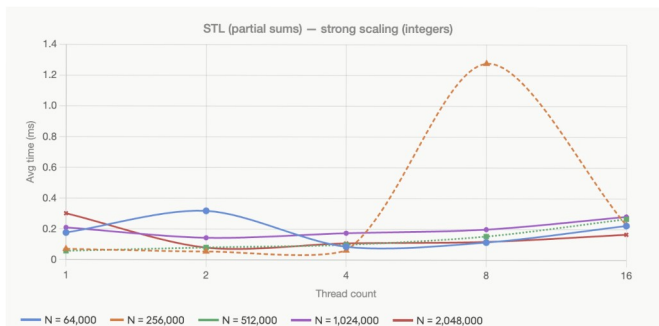


A6: C++ and OpenMP Parallel Reductions – CS6013 – 4/29/26 – Jon Pulsipher

Strong Scaling

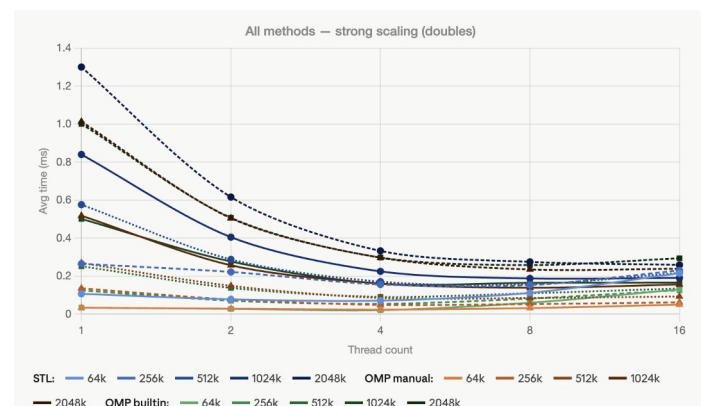
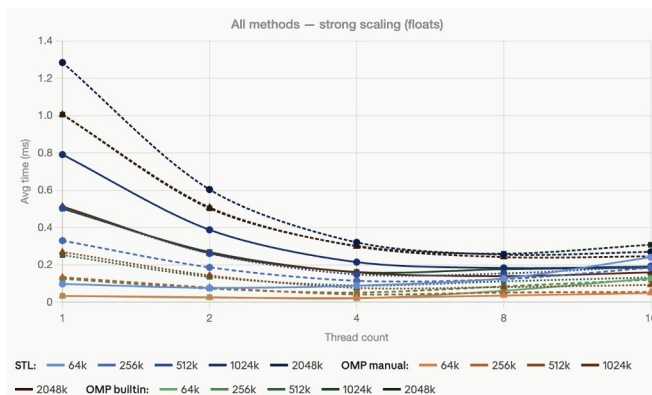
To test strong scaling, I used arrays starting at size 64,000 and increasing by powers of 2 to 2,048,000. I did not chart lower values because when I measured them they were noisier—a result that would be expected given how quickly a computer can process that amount of data such that runtime ends up being more defined by everything else happening on the computer. *When I tested lower array sizes, I did not consistently see timing that fit my expectation that more cores would increase speed.* I used thread counts from 1 to 16, also increasing by powers of 2.



Do the curves match the ideal case?

Overall they do, up to 4 cores. This fits the hardware of the macbook on which these tests were run which has 4 performance cores. It does have 6 more efficiency cores, but when thread count increases to include those efficiency cores, runtime increases for all approaches. This could suggest that any benefit from including those cores is outweighed by the overhead of more threads. There is some anomalous data that does not fit this trend, especially in the 1st STL (partial sums) graph. Fittingly, it is more pronounced for the smaller array sizes: 64,000 and 256,000. The N = 256,000, thread count 8 spike is such an outlier that it may be explained by an interruption of some expensive process elsewhere on the computer.

Comparing integers to floats and doubles:

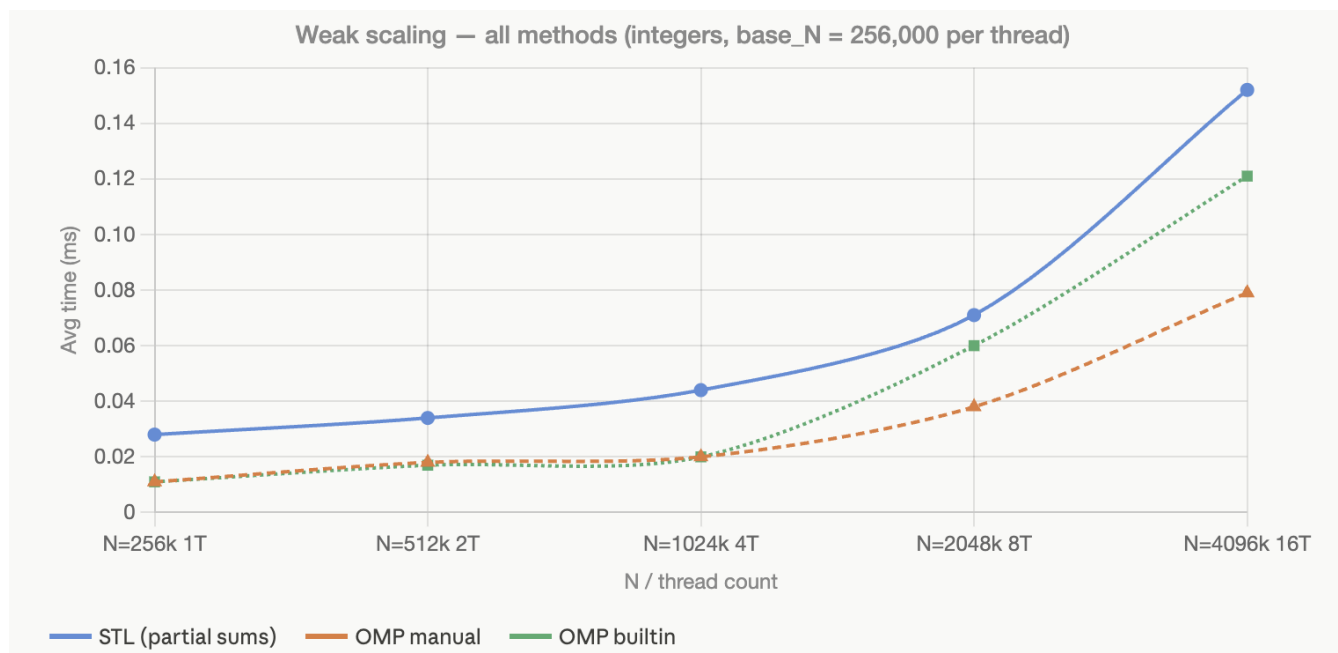


Floats and doubles returns very similar timings and curves that different from integers primarily by taking 2-3x the time and more consistently showing improved times at threadcounts of 8 and 16. The main difference here may just be the increase runtime – it allows the added overhead of threads to be outweighed by work those threads do.

What explains these differences? The int to double difference can be explained by size: ints are 4-bytes while doubles are 8 bytes. Floats are also 4 bytes though they perform similarly to doubles, but they do use different pathways for addition which is known to be slower.

One anomalous bit of this data is that the macbook this is running only has 10 cores. I still wanted to include this data to see the effect of having more threads than your hardware contains. Effectively, 16 threads is using 10 cores at max, which may be we occasionally see an improvement from 8 to 16, but not always.

Weak Scaling



Does the curve match the ideal case?

No, because runtime does not remain constant. But as N is scaling runtime is increasing, but not proportionally – from 1 to 4 cores we see runtime doubling as N quadruples. As before, this effectiveness of this scaling drops after 4 cores when we run out of performance cores. Another reason we are not seeing a flat curve is that increasing thread count still increases overhead.

Comparison

Which of the three methods above performs best on each test?

Overall, OMP built-in performed most efficiently, with OMP manual coming in second. This is to be expected from an optimized library and, in the case of OMP built-in, the reduction function which is optimized to do operations like summing an array.